Processi periodici

Indice dei contenuti

- 1. Testo
  - 1.1. Script Python
  - 1.2. Service
  - 1.3. Timer
- 2. Suggerimenti
  - 2.1. Manuali di riferimento di sistema
  - 2.2. Script Python
    - 2.2.1. Esecuzione del programma
    - 2.2.2. Modalità interattiva
    - 2.2.3. Documentazione
    - 2.2.4. Struttura del programma
    - 2.2.5. Ricorsione
    - 2.2.6. Ricerca di stringhe
  - 2.3. Service
    - 2.3.1. Struttura del file unit
    - 2.3.2. Posizione del file unit
    - 2.3.3. Gestione del service
    - 2.3.4. Log
  - 2.4. Timer
    - 2.4.1. Struttura del file unit
    - 2.4.2. Posizione del file unit
    - 2.4.3. Verifica del timer
    - 2.4.4. Gestione del timer
- 3. Soluzione
- Licenze

1. Testo

1.1. Script Python

Scrivi uno script Python che analizzi i file di log in una directory specificata e nelle sue sottodirectory, estraendo le righe che contengono una stringa specificata e scrivendole in file di output in una directory di backup chiamata `backup`, creata (se necessario) in `~`. Nella tua home directory, crea una directory chiamata `log-extractor` e, al suo interno, un file chiamato `app.py`, usando questo template:

```python
# nome e cognome:
# matricola:
#
# path:

import argparse
import os
import sys

def main():
    pass


if __name__ == "__main__":
    main()
```

Lo script deve accettare esattamente due argomenti da riga di comando, analizzati tramite il modulo `argparse`. Il primo argomento, `--path`, è una stringa obbligatoria che specifica il percorso assoluto della directory da analizzare. Il secondo argomento, `--pattern`, è una stringa obbligatoria che specifica il pattern da cercare all'interno dei file di log, ovvero i file con estensione `.log`.

Dopo il parsing, valida entrambi gli input: verifica che il path sia assoluto (`os.path.isabs`), che esista (`os.path.exists`) e che sia una directory (`os.path.isdir`); verifica che `--pattern` sia una stringa non vuota. Se una qualsiasi verifica fallisce, stampa un messaggio di errore esplicativo sullo standard error (`print`) ed esci con un codice di stato non zero (`sys.exit`).

Dopo la validazione, assicurati che la directory `backup` esista in `~` (`os.path.expanduser` , `os.makedirs`). Esplora ricorsivamente l'albero di directory al path fornito (`os.listdir` , `os.path.join` , `os.path.isdir`). Per ogni file di log incontrato (`os.path.isfile` , `str.endswith`), verifica se contiene il pattern specificato leggendone il contenuto (`open`). Se il pattern viene trovato, estrai le righe corrispondenti e scrivile in un file con lo stesso nome nella directory `~/backup` (`open` ,

`file.writelines`), stampando un messaggio di log sullo standard output (`print`) che indichi il numero di righe scritte e il nome del file di destinazione.

Ad esempio, eseguendo:

```
$ python ~/log-extractor/app.py \
    --path ~/logs \
    --pattern ERROR
```

lo script estrarrà tutte le righe contenenti la stringa `ERROR` dai file con estensione `.log` in `~/logs` e nelle sue sottodirectory, scrivendole nei file corrispondenti nella cartella `~/backup` .

1.2. Service

Crea un'unità service denominata `log-extractor.service` nell'istanza utente di `systemd` . Configurala per eseguire `~/log-extractor/app.py` con gli argomenti `--path %h/logs` e `--pattern ERROR` . Usa questo template:

```
# nome e cognome:
# matricola:
#
# path:
```

1.3. Timer

Crea un'unità timer denominata `log-extractor.timer` nell'istanza utente di `systemd` . Configurala per attivare `log-extractor.service` ogni lunedì e venerdì alle 02:00. Usa questo template:

```
# nome e cognome:
# matricola:
#
# path:
#
# comando per abilitare il timer:
# comando per avviare il timer:
```

2. Suggerimenti

2.1. Manuali di riferimento di sistema

`man` visualizza i manuali di riferimento di sistema. Ad esempio, `man:systemctl(1)` si traduce in:

```
$ man 1 systemctl
```

2.2. Script Python

2.2.1. Esecuzione del programma

Per eseguire un programma Python:

```
$ python3 <tuo_programma>.py
```

> ⚠ **Warning**
>
> A seconda del sistema in uso, potrebbe essere necessario usare `python` oppure `python3`.

2.2.2. Modalità interattiva

La modalità interattiva avvia un ciclo read-eval-print (REPL) in cui è possibile digitare espressioni Python e vederne immediatamente i risultati. È utile per sperimentare con funzioni e moduli prima di scrivere un programma completo.

Per avviare la modalità interattiva:

```
$ python3
```

2.2.3. Documentazione

`help()` mostra la documentazione di moduli, funzioni, classi e keyword nell'interprete interattivo Python. Ad esempio, per leggere la documentazione del modulo `argparse`:

```
$ python3
>>> import argparse
>>> help(argparse)
```

Per ottenere aiuto su una funzione specifica:

```
$ python3
>>> import os.path
>>> help(os.path.isabs)
```

2.2.4. Struttura del programma

Il seguente esempio estende il template della §1.1 con commenti che corrispondono a ciascun passo descritto nel testo.

```python
# nome e cognome:
# matricola:
#
# path:

import argparse
import os
import sys


def find(filename, pattern):
    # leggi il file, restituisci le righe corrispondenti
    pass


def dump(filename, lines):
    # scrivi le righe nel file
    pass


def walk(log_dir, backup_dir, pattern):
    # esplorazione ricorsiva
    pass


def main():
    # parsing degli argomenti

    # validazione

    # assicura che la directory di backup esista

    # scansione
    pass


if __name__ == "__main__":
    main()
```

Tip

`os.makedirs` solleva `FileExistsError` se la directory esiste già. Passa `exist_ok=True` per ignorare silenziosamente la creazione quando la directory è già presente.

2.2.5. Ricorsione

Il seguente esempio elenca ricorsivamente tutti i percorsi dei file in un albero di directory.

```
import os


def walk(basepath):
    for filename in os.listdir(basepath):
        path = os.path.join(basepath, filename)
        if os.path.isfile(path):
            print(path)
        elif os.path.isdir(path):
            walk(path)
```

2.2.6. Ricerca di stringhe

Il seguente esempio legge tutte le righe da un file e mantiene solo quelle che contengono una determinata stringa.

```
with open("server.log", "r") as f:
    lines = f.readlines()

matching = [line for line in lines if "timeout" in line]
```

2.3. Service

2.3.1. Struttura del file unit

Il seguente esempio estende il template della §1.2 con commenti che corrispondono a ciascuna sezione del file unit.

```
# nome e cognome:
# matricola:
#
# path:

[Unit]
# descrizione leggibile dall'utente

[Service]
# come eseguire il service
```

Un esempio di file unit per un service è disponibile qui. Le direttive disponibili sono documentate in `man:systemd.unit(5)` (per `[Unit]`) e `man:systemd.service(5)` (per `[Service]`).

> **Tip**
>
> `%h` si espande nella home directory dell'utente che esegue il service. Ad esempio, `WorkingDirectory=%h/log-extractor` si espande in `WorkingDirectory=/home/ubuntu/log-extractor` .

> **Tip**
>
> Aggiungi `Environment=PYTHONUNBUFFERED=1` a `[Service]` in modo che l'output di `print` compaia immediatamente in `journalctl` .

> **Warning**
>
> Questo file unit per il service non ha una sezione `[Install]` . L'unità timer è responsabile dell'attivazione del service, quindi il service non ha bisogno di essere abilitato o avviato autonomamente all'avvio.

2.3.2. Posizione del file unit

Le directory in cui `systemd` legge i file unit sono documentate in `man:systemd.unit(5)` . La directory consigliata per le unità utente è `~/.config/systemd/user` . Se non esiste, creala:

```
$ mkdir -p ~/.config/systemd/user
```

Poi copia il file unit lì:

```
$ cp ~/log-extractor/log-extractor.service ~/.config/systemd/user
```

2.3.3. Gestione del service

I comandi per controllare `systemd` sono elencati qui. Aggiungi l'opzione `--user` per controllare l'istanza utente invece di quella di sistema. Vedi anche `man:systemctl(1)` .

Dopo aver copiato il file unit:

- ☐ Elenca i file unit per verificare che `systemd` veda il nuovo file (`list-unit-files`)
- ☐ Ispeziona il file unit per verificarne il contenuto (`cat`)
- ☐ Ricarica i file unit in modo che `systemd` recepisca le modifiche (`daemon-reload`)
- ☐ Avvia il service (`start`)
- ☐ Controlla lo stato per confermare che sia stato eseguito correttamente (`status`)

2.3.4. Log

`journalctl` interroga il journal di `systemd` . Aggiungi l'opzione `--user` per leggere i log dall'istanza utente. Ad esempio:

```
$ journalctl --user -u log-extractor.service
```

Vedi anche `man:journalctl(1)` .

2.4. Timer

2.4.1. Struttura del file unit

Il seguente esempio estende il template della §1.3 con commenti che corrispondono a ciascuna sezione del file unit.

```
# nome e cognome:
# matricola:
#
# path:
#
# comando per abilitare il timer:
# comando per avviare il timer:

[Unit]
# descrizione leggibile dall'utente

[Timer]
# quale service attivare
# pianificazione (espressione calendario)

[Install]
# avvio al boot
```

Le direttive disponibili sono documentate in `man:systemd.timer(5)` (per `[Timer]`) e `man:systemd.unit(5)` (per `[Unit]` e `[Install]`). I tipi di timer e le espressioni calendario sono trattati qui.

> **Tip**
>
> Usa `WantedBy=timers.target` nella sezione `[Install]` . `timers.target` è un target dedicato che raccoglie tutte le unità timer e viene attivato in anticipo durante il boot. È preferibile a `default.target` , che riguarda il portare il sistema al suo normale stato operativo, non la gestione dei timer.

2.4.2. Posizione del file unit

I file unit del timer vanno nella stessa directory dei file unit del service: `~/.config/systemd/user` .

```
$ cp ~/log-extractor/log-extractor.timer ~/.config/systemd/user
```

2.4.3. Verifica del timer

`systemd-analyze calendar` analizza le espressioni di calendario e calcola quando scatteranno la prossima volta. Usalo per verificare la tua espressione `OnCalendar` prima di distribuire il timer.

```
$ systemd-analyze calendar 'Wed *-*-* 14:00' --iterations=3
  Original form: Wed *-*-* 14:00
Normalized form: Wed *-*-* 14:00:00
    Next elapse: Wed 2025-05-21 14:00:00 UTC
       From now: 1 day left
   Iteration #2: Wed 2025-05-28 14:00:00 UTC
       From now: 8 days left
   Iteration #3: Wed 2025-06-04 14:00:00 UTC
       From now: 15 days left
```

Per ulteriori informazioni sulle espressioni temporali, vedi qui.

2.4.4. Gestione del timer

Dopo aver copiato il file unit:

- [ ] Elenca i file unit per verificare che `systemd` veda il nuovo file (`list-unit-files`)
- [ ] Ispeziona il file unit per verificarne il contenuto (`cat`)
- [ ] Ricarica i file unit in modo che `systemd` recepisca le modifiche (`daemon-reload`)
- [ ] Avvia il timer (`start`)
- [ ] Controlla lo stato per confermare che sia attivo e in attesa (`status`)
- [ ] Elenca i timer attivi per vedere il prossimo orario di scatto (`list-timers`)
- [ ] Abilita il timer in modo che si avvii al boot (`enable`)
- [ ] Riavvia il sistema
- [ ] Controlla lo stato per confermare che sia sopravvissuto al riavvio (`status`)

3. Soluzione

Questo esercizio è stato proposto il 8 settembre 2025.

Licenze

Contenuto	Licenza
Codice	MIT License
Testo	Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International